

src/well_simulator.py

359 lines

```
1 """
2 well_simulator.py
3 =====
4
5 Physics-informed simulator of a SINGLE NATURALLY FLOWING OIL WELL with one
6 production choke.
7
8 >>> IMPORTANT NOTE ON PROVENANCE <<<
9 The hackathon problem statement says a simulator "will be provided". No
10 simulator file was made available to us, so this module is a physics-based
11 STAND-IN built directly from the process description in the problem statement
12 (reservoir -> bottom hole -> tubing -> wellhead -> choke -> flowline ->
13 manifold -> separator).
14
15 It exposes exactly the interface named in the problem statement:
16
17 Q, WHP, FLP, BHP = simulator.step(choke_position)
18
19 and nothing else is used by the controller (see `simulator_interface.py`), so
20 the official simulator can be substituted with no controller changes.
21
22 -----
23 MODEL
24 -----
25 At every control interval the *steady-state* operating point implied by the
26 commanded choke opening is solved from four simultaneous relations, and each
27 measured variable is then moved towards it through a first-order lag.
28
29 1. Reservoir inflow performance (linear IPR)
30  $BHP = P_{res} - Q / PI$ 
31 Drawdown grows linearly with rate; more flow means less bottom hole
32 pressure. A linear IPR (constant productivity index) is the standard
33 first-order description above the bubble point and is explicitly adequate
34 for this scope.
35
36 2. Production tubing hydraulics
37  $WHP = BHP - dP_{static} - k_{fric} * Q^2$ 
38 The static head (fluid column) is constant for a fixed fluid gradient; the
39 friction term is turbulent and therefore grows with the square of rate.
40
41 3. Production choke (orifice / valve equation)
42  $Q = C_v * f(u) * \sqrt{WHP - FLP}$ ,  $f(u) = (u/100)^{1.5}$ 
43 The single most important relation in the model: flow through the choke
44 depends on BOTH the opening AND the pressure drop across the valve. Q is
45 *not* a direct function of choke position alone. Opening the choke
46 increases Q, which increases drawdown, which lowers WHP, which reduces the
47 available dP - a self-limiting negative feedback that is exactly what makes
48 the process nonlinear and interesting to control.
49
50 4. Flowline backpressure
51  $FLP = FLP_{base} + k_{flowline} * Q$ 
52 A separator/manifold baseline plus a mild rise with throughput.
53
54 Because (1)-(4) are coupled, the steady state is found by solving the scalar
55 residual
56
57  $g(Q) = C_v * f(u) * \sqrt{\max(WHP(Q) - FLP(Q), 0)} - Q = 0$ 
58
59 which is strictly decreasing in Q (raising Q lowers WHP and raises FLP, so it
60 lowers the right-hand side while raising the left). A monotone residual means
61 a bracketed bisection/Brent solve is guaranteed to converge - there is no way
62 for the simulator to produce a NaN.
63
64 5. Dynamics and measurement
65 Every output is passed through a first-order lag with its own time constant
66 (BHP slowest, flowline fastest), then corrupted with small Gaussian sensor
67 noise. This makes the plant genuinely dynamic, so the controller has to
```

```

68 *predict* rather than merely invert a static curve.
69
70 -----
71 ASSUMPTIONS (per the problem statement)
72 -----
73 * single naturally flowing well, single production choke, no artificial
74 lift, no gas lift / ESP optimisation
75 * no facility-network interaction, constant reservoir properties,
76 constant GOR and water cut
77 * isothermal hydraulics (WHT is reported but is informational only)
78 * the choke responds to the commanded position within one control interval
79 (actuator dynamics are fast relative to Ts = 1 h)
80
81 All parameters live in `config.PlantParams` and are easy to retune.
82 """
83
84 from __future__ import annotations
85
86 from typing import Dict, List, Tuple
87
88 import numpy as np
89 import pandas as pd
90
91 try: # works both as a package and a script
92     from .config import PLANT, SEED, TS_HOURS, PlantParams
93 except ImportError: # pragma: no cover
94     from config import PLANT, SEED, TS_HOURS, PlantParams
95
96
97 class WellSimulator:
98     """
99     A single naturally flowing oil well with one production choke.
100
101     Parameters
102     -----
103     params : PlantParams, optional
104     Physics parameters (defaults to `config.PLANT`).
105     seed : int, optional
106     Seed for the sensor-noise RNG; fixed by default for reproducibility.
107     noise : bool, optional
108     Set False for a noise-free plant (used by the step-test analysis to
109     show the underlying dynamics cleanly, and by unit tests).
110     ts : float, optional
111
112     Control interval in hours (default 1.0).
113
114     Examples
115     -----
116     >>> sim = WellSimulator()
117     >>> Q, WHP, FLP, BHP = sim.step(30.0)
118     """
119     def __init__(
120         self,
121         params: PlantParams = PLANT,
122         seed: int = SEED,
123         noise: bool = True,
124         ts: float = TS_HOURS,
125     ) -> None:
126         self.p = params
127         self.ts = ts
128         self.noise_on = noise
129         self._seed = seed
130         self.reset()
131
132     # -----
133     # Steady-state physics
134     # -----
135     def _bhp_of_q(self, q: float) -> float:
136         """Linear IPR: bottom hole pressure falls as rate rises."""
137         return self.p.p_res - q / self.p.pi_index

```

```

138
139 def _whp_of_q(self, q: float) -> float:
140     """Tubing: static head is constant, friction grows with Q^2."""
141     dp_static = self.p.tvd_ft * self.p.grad_psi_per_ft
142     return self._bhp_of_q(q) - dp_static - self.p.k_fric * q * q
143
144 def _flp_of_q(self, q: float) -> float:
145     """Flowline: baseline separator backpressure plus friction with rate."""
146     return self.p.flp_base + self.p.k_flowline * q
147
148 def _choke_char(self, u: float) -> float:
149     """Valve characteristic f(u), 0 at shut, 1 at fully open."""
150     u = float(np.clip(u, 0.0, 100.0))
151     return (u / 100.0) ** self.p.choke_exp
152
153 def _residual(self, q: float, u: float) -> float:
154     """g(Q) = choke-passing flow at this Q - Q. Strictly decreasing."""
155     dp = self._whp_of_q(q) - self._flp_of_q(q)
156     dp = max(dp, 0.0)
157     return self.p.cv * self._choke_char(u) * np.sqrt(dp) - q
158
159 def steady_state(self, u: float) -> Dict[str, float]:
160     """
161     Solve the coupled steady state for a given choke opening.
162
163     Returns the noise-free equilibrium {Q, WHP, FLP, BHP, WHT, AP}.
164     Uses bisection on the monotone residual g(Q); guaranteed to converge,
165     so no NaN can ever leave this function.
166     """
167     u = float(np.clip(u, 0.0, 100.0))
168
169     if self._choke_char(u) <= 0.0: # fully shut -> no flow at all
170         q = 0.0
171     else:
172         # Upper bracket: the rate at which the choke differential pressure
173         # collapses to zero (no more driving force). g(q_hi) = -q_hi < 0.
174         lo, hi = 0.0, self._q_at_zero_dp()
175         if self._residual(lo, u) <= 0.0:
176             q = 0.0
177         else:
178             for _ in range(200): # bisection: ~1e-13 relative in 200
179                 mid = 0.5 * (lo + hi)
180                 if self._residual(mid, u) > 0.0:
181                     lo = mid
182                 else:
183                     hi = mid
184             q = 0.5 * (lo + hi)
185
186     return {
187         "Q": q,
188         "WHP": self._whp_of_q(q),
189         "FLP": self._flp_of_q(q),
190         "BHP": self._bhp_of_q(q),
191         "WHT": self.p.wht_base + self.p.wht_gain * q,
192         "AP": self.p.annulus_pressure,
193     }
194
195 def _q_at_zero_dp(self) -> float:
196     """
197     Largest physically meaningful rate: the Q at which WHP - FLP = 0.
198     Solves a*Q^2 + b*Q - c = 0 from the tubing/IPR/flowline relations.
199     """
200     a = self.p.k_fric
201     b = 1.0 / self.p.pi_index + self.p.k_flowline
202     c = (
203         self.p.p_res
204         - self.p.tvd_ft * self.p.grad_psi_per_ft
205         - self.p.flp_base
206     )
207     if c <= 0.0: # well cannot flow at all

```

```

208 return 0.0
209 return (-b + np.sqrt(b * b + 4.0 * a * c)) / (2.0 * a)
210
211 # -----
212 # Dynamics
213 # -----
214 def _lag(self, current: float, target: float, tau: float) -> float:
215     """Exact discretisation of a first-order lag over one interval."""
216     alpha = 1.0 - np.exp(-self.ts / tau)
217     return current + alpha * (target - current)
218
219 # -----
220 # Public API (this is the entire contract the controller relies on)
221 # -----
222 def step(self, choke_position: float) -> Tuple[float, float, float, float]:
223     """
224     Advance the well by one control interval.
225
226     Parameters
227     -----
228     choke_position : float
229     Commanded choke opening [%], clipped internally to 0-100.
230
231     Returns
232     -----
233     (Q, WHP, FLP, BHP) : noisy measurements after this interval.
234     """
235     u = float(np.clip(choke_position, 0.0, 100.0))
236     ss = self.steady_state(u)
237
238     # First-order lag on every state.
239     self._x["Q"] = self._lag(self._x["Q"], ss["Q"], self.p.tau_q)
240     self._x["WHP"] = self._lag(self._x["WHP"], ss["WHP"], self.p.tau_whp)
241     self._x["FLP"] = self._lag(self._x["FLP"], ss["FLP"], self.p.tau_flp)
242     self._x["BHP"] = self._lag(self._x["BHP"], ss["BHP"], self.p.tau_bhp)
243     self._x["WHT"] = self._lag(self._x["WHT"], ss["WHT"], self.p.tau_wht)
244     self._x["AP"] = ss["AP"]
245
246     self.t += self.ts
247     self.u = u
248
249     meas = self._measure()
250     self._log(meas)
251     return meas["Q"], meas["WHP"], meas["FLP"], meas["BHP"]
252
253 def _measure(self) -> Dict[str, float]:
254     """Apply sensor noise and physical floors to the internal state."""
255     n = self._rng.normal
256     p = self.p
257     if self.noise_on:
258         m = {
259             "Q": self._x["Q"] + n(0.0, p.noise_q),
260             "WHP": self._x["WHP"] + n(0.0, p.noise_whp),
261             "FLP": self._x["FLP"] + n(0.0, p.noise_flp),
262             "BHP": self._x["BHP"] + n(0.0, p.noise_bhp),
263             "WHT": self._x["WHT"] + n(0.0, p.noise_wht),
264             "AP": self._x["AP"] + n(0.0, p.noise_ap),
265         }
266     else:
267         m = dict(self._x)
268     m["Q"] = max(m["Q"], 0.0) # a naturally flowing well cannot backflow
269     return m
270
271 def reset(self) -> Tuple[float, float, float, float]:
272     """
273     Return the well to shut-in conditions (choke closed, no flow, pressures
274     equalised to reservoir) and clear the history log.
275     """

```

```

276 self._rng = np.random.default_rng(self._seed)
277 ss = self.steady_state(0.0)
278 self._x = dict(ss)
279 self.t = 0.0
280 self.u = 0.0
281 self.history: List[Dict[str, float]] = []
282 meas = self._measure()
283 self._log(meas)
284 return meas["Q"], meas["WHP"], meas["FLP"], meas["BHP"]
285
286 # -----
287 # History logging
288 # -----
289 def _log(self, meas: Dict[str, float]) -> None:
290     self.history.append(
291         {
292             "time_h": self.t,
293             "choke_pct": self.u,
294             "Q": meas["Q"],
295             "WHP": meas["WHP"],
296             "FLP": meas["FLP"],
297             "BHP": meas["BHP"],
298             "WHT": meas["WHT"],
299             "AP": meas["AP"],
300             # Noise-free internal state, for analysis only. The controller
301             # never reads these columns.
302             "Q_true": self._x["Q"],
303             "WHP_true": self._x["WHP"],
304             "FLP_true": self._x["FLP"],
305             "BHP_true": self._x["BHP"],
306         }
307     )
308
309 def to_dataframe(self) -> pd.DataFrame:
310     """Full run history as a tidy DataFrame."""
311     return pd.DataFrame(self.history)
312
313 # -----
314 # Analysis helpers (NOT used by the controller)
315 # -----
316 def steady_state_curve(self, u_grid=None) -> pd.DataFrame:
317     """Noise-free steady-state operating curve, for plots and analysis."""
318     if u_grid is None:
319         u_grid = np.linspace(0.0, 100.0, 201)
320     rows = [{"choke_pct": u, **self.steady_state(u)} for u in u_grid]
321     return pd.DataFrame(rows)
322
323 def max_safe_rate(self, limits, resolution: float = 0.01) -> Tuple[float, float]:
324     """
325     Ground-truth maximum steady-state rate that keeps every pressure inside
326     `limits`, and the choke opening that achieves it.
327
328     `resolution` is the choke-position step in percent (default 0.01 %).
329
330     Used ONLY to check the controller's answer in Scenario C - the
331     controller itself never calls this.
332     """
333     best_q, best_u = 0.0, 0.0
334     for u in np.arange(0.0, 100.0 + resolution, resolution):
335         ss = self.steady_state(u)
336         ok = (
337             limits.whp_min <= ss["WHP"] <= limits.whp_max
338             and limits.flp_min <= ss["FLP"] <= limits.flp_max
339             and limits.bhp_min <= ss["BHP"] <= limits.bhp_max
340         )
341         if ok and ss["Q"] > best_q:
342             best_q, best_u = ss["Q"], u
343     return best_q, best_u
344
345
346 if __name__ == "__main__": # quick smoke test / sanity print

```

```
347 from config import LIMITS
348
349 sim = WellSimulator(noise=False)
350 print(f"{'u [%]':>7} {'Q [bbl/hr]':>11} {'WHP':>8} {'FLP':>8} {'BHP':>8}")
351 for u in [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]:
352     ss = sim.steady_state(u)
353     print(
354         f"{u:7.0f} {ss['Q']:11.1f} {ss['WHP']:8.1f} "
355         f"{ss['FLP']:8.1f} {ss['BHP']:8.1f}"
356     )
357     q_max, u_max = sim.max_safe_rate(LIMITS)
358     print(f"\nMaximum SAFE steady-state rate: {q_max:.1f} bbl/hr at u = {u_max:.1f} %")
359
```